

POI Graphs under Spatial Noise and Temporal Drift

Klyuchnikova Ulyana, Urazmetova Margarita

Email: udklyuchnikova@edu.hse.ru, mrurazmetova@edu.hse.ru,

Abstract—Next Point-of-Interest (POI) recommendation is a central task in location-based social networks, aiming to predict a user’s future destinations from historical mobility trajectories. Recent advances in graph neural networks and sequential modeling have significantly improved recommendation accuracy by incorporating spatial transitions, contextual information, and high-order behavioral dependencies. However, real-world mobility data are often affected by GPS inaccuracies, sparse observations, and temporal distribution shifts caused by seasonal behavior changes, venue popularity dynamics, and evolving user preferences. Existing evaluations are commonly conducted on heavily preprocessed datasets with occurrence-based filtering and static train-test assumptions, making the robustness of modern POI recommendation methods under realistic conditions insufficiently understood.

This project investigates the robustness of next-POI recommendation methods under spatial perturbation and temporal drift. Rather than proposing a new state-of-the-art architecture, we develop a systematic evaluation framework for comparing representative sequential and graph-based recommendation models under varying preprocessing settings and controlled spatial perturbations. Experiments will be conducted on publicly available Foursquare and Gowalla datasets using standard ranking metrics including Acc@k, MAP@k and MRR.

The goal of this work is to provide practical insights into the reliability and deployment readiness of modern POI recommendation systems in noisy and non-stationary environments.

Index Terms—POI recommendation, GNN, spatial noise, robustness evaluation, comparative analysis

I. INTRODUCTION

LOCATION-BASED social networks (LBSNs) such as Foursquare and Gowalla have enabled the large-scale collection of human mobility data through user check-ins and location-sharing services. These data have driven substantial research interest in next Point-of-Interest (POI) recommendation, a task that aims to predict the next location a user is likely to visit based on historical trajectories. Accurate next-POI recommendation has important applications in urban computing, personalized services, tourism, advertising, and intelligent transportation systems.

Early approaches to next-POI recommendation primarily relied on Markov chains and recurrent neural networks to model sequential mobility behavior. More recently, graph-based methods have become dominant due to their ability to capture complex transition patterns and collaborative relations between POIs. Graph neural networks (GNNs), attention mechanisms, and Transformer-based architectures have demonstrated strong performance by modeling spatial proximity, trajectory transitions, and contextual dependencies across user mobility graphs. Recent work has further explored multi-graph fusion and contrastive learning to improve representation quality under sparse check-in data.

Although recent POI recommendation methods incorporate sophisticated graph structures and self-supervised learning objectives, relatively little attention has been devoted to robustness evaluation under spatial perturbation and temporal drift. Existing studies typically employ random or short-horizon train-test splits and evaluate models on clean benchmark datasets, which may overestimate real-world generalization performance or result in simple difference in training data between models. As a result, it remains unclear which architectural mechanisms contribute most to robustness under noisy and non-stationary conditions.

This work addresses this gap through a systematic comparative study of representative sequential and graph-based next-POI recommendation methods under controlled spatial and temporal distribution shifts. Instead of introducing another complex recommendation architecture, the primary goal of this project is to evaluate how existing methods behave when deployed with varying noise levels.

II. RELATED WORK

Contrastive learning has recently emerged as an effective self-supervised learning strategy for recommendation systems. The general idea is to generate multiple augmented views of graph-structured

data and learn representations by maximizing agreement between positive pairs while separating negative samples.

Several foundational methods introduced contrastive objectives for graph recommendation. SGL [1] applies node and edge dropout to generate augmented graph views, while SimGCL [2] introduces perturbations directly in the embedding space to improve robustness and representation consistency. NCL [3] further incorporates neighborhood-enriched contrastive objectives to capture structural information more effectively.

In the POI recommendation domain, contrastive learning is commonly used to alleviate sparsity and improve multi-view representation learning. CLLP [4] combines spatiotemporal preference modeling with contrastive objectives, while MGMCL [5] aligns representations across multiple mobility graphs through local and global contrastive learning strategies.

Although these approaches improve representation quality and generalization, contrastive augmentations are typically designed for representation learning rather than systematic robustness evaluation. In particular, the effects of controlled spatial perturbation and temporal distribution shift on next-POI recommendation models remain insufficiently explored.

III. DATASETS AND PREPROCESSING

We use three real-world check-in datasets:

- **TSMC2014_NYC**: 227,428 check-ins, 21,000 POIs, 1,083 users (Foursquare, Apr 2012–Feb 2013) [6]
- **TSMC2014_TKY**: 537,703 check-ins, 18,000 POIs, 2,293 users (Foursquare, Apr 2012–Feb 2013) [6]
- **Gowalla (global)**: 6.4M check-ins, 1.28M POIs, 107,092 users with check-ins (Feb 2009–Oct 2010) [7]

A. Exploratory Data Analysis

a) *Gowalla*: We analyzed the full SNAP Gowalla check-in dataset before graph construction. The raw file contains check-ins with user IDs, timestamps, coordinates, and POI IDs over the period February 2009–October 2010.

SNAP Gowalla does not include POI names or semantic categories, so we enriched POIs with

TABLE I
CLEANED GOWALLA DATASET SUMMARY

Statistic	Value
Raw check-ins	6,442,892
Cleaned check-ins	6,442,262
Users	107,092
POIs	1,280,956
Category coverage by POI	96.8%
Category coverage by check-in	97.3%
POI name coverage by POI	0.5%
POI name coverage by check-in	0.2%

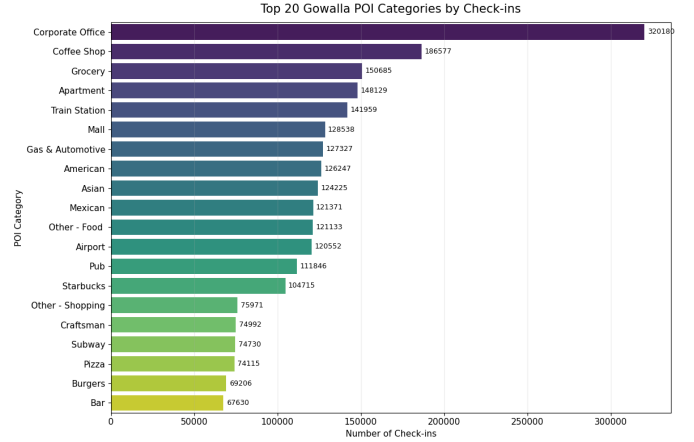


Fig. 1. Top Gowalla POI categories by number of check-ins.

Figshare Gowalla metadata [8]. Category metadata covers 96.8% of POIs and 97.3% of check-ins. POI names cover only 0.5% of POIs and 0.2% of check-ins; therefore, the analysis focuses mainly on POI categories.

Fig. 1 shows the most frequent POI categories. The leading categories include corporate offices, coffee shops, grocery stores, apartments, train stations, malls, and transportation-related venues. This indicates that Gowalla contains heterogeneous urban mobility patterns rather than one dominant type of location.

Fig. 2 shows row-normalized hourly profiles for the most frequent categories. Each row sums to one, so the figure compares the daily rhythm within each category. The profiles differ substantially across categories, which confirms that time-of-day information is relevant for next-POI modeling.

The POI distribution is highly long-tailed (Fig. 3). In the cleaned dataset, 17,886 POIs cover 25% of check-ins, 101,432 cover 50%, 346,882 cover 75%, and 709,806 cover 90%. This confirms strong spar-

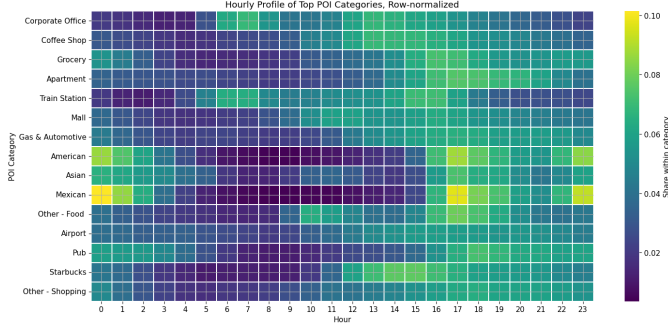


Fig. 2. Hourly profile of top POI categories, row-normalized within each category.

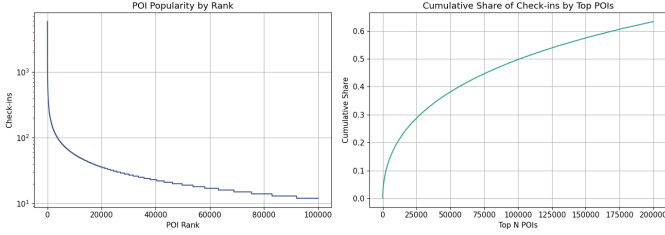


Fig. 3. POI popularity long tail and cumulative check-in coverage.

sity at the POI level and motivates filtering, graph augmentation, and careful treatment of infrequent locations.

Additional EDA results show that user activity is also uneven: most users have relatively few check-ins, while a small subset is much more active. Repeat-visit analysis shows that trajectories contain both returns to previously visited POIs and exploration of new locations. We also constructed consecutive POI transitions and measured transition time gaps and distances. These results indicate that transition graph construction should account for temporal windows and spatial distance, rather than treating all consecutive check-ins as equally meaningful mobility transitions.

Fig. 4 summarizes temporal drift. Monthly active users and active POIs change substantially over time, while the overlap of top POIs across adjacent months is not fully stable. This supports chronological train/validation/test splitting instead of random splitting, since future mobility behavior may differ from past behavior.

Overall, the EDA shows three properties relevant to our study: high category coverage, strong long-tail sparsity, and temporal drift. These properties motivate robustness evaluation under spatial noise and temporal distribution shift. Additional EDA

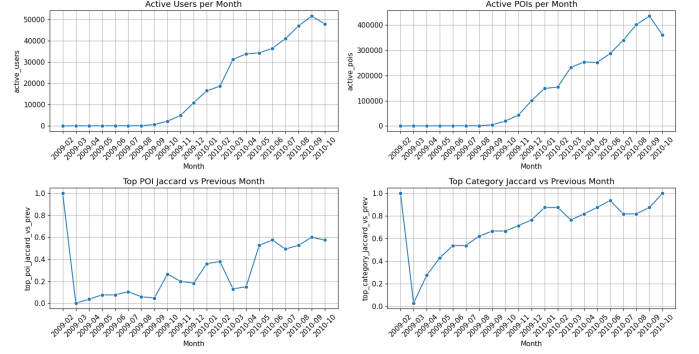


Fig. 4. Temporal drift in monthly Gowalla activity and top-entity stability.

TABLE II
BASIC DATASET STATISTICS

Metric	New York City	Tokyo
Total Check-ins	227,428	573,598
Unique Users	1,083	2,293
Unique Venues	38,333	61,858
Unique Categories	251	247
Avg Check-ins per User	210.0	250.2
Median Check-ins per User	153	173
Min Check-ins per User	100	100
Max Check-ins per User	2,697	2,991

tables, preprocessing checks, and visualizations are provided in the supplementary notebook [9].

b) Foursquare: We also considered Foursquare [6] which is a common LBSN dataset. This dataset contains check-ins in NYC (New York City) and TKY (Tokyo) collected for about 10 month (from 12 April 2012 to 16 February 2013). Each check-in is associated with its time stamp, its GPS coordinates and its semantic meaning (represented by fine-grained venue-categories).

The general statistics (Table II) show that Tokyo contains roughly twice as much data as NYC, with both user count and total check-ins approximately doubling. Foursquare appears to have applied some filtering based on minimum check-ins per user, though the threshold appears to be relatively low, so this should not pose a concern for analysis.

A more detailed examination of user activity distributions (Fig. 5) reveals that NYC and Tokyo exhibit rather similar patterns, although Tokyo users appear slightly more active. Both datasets contain anomalous users with over 2,500 check-ins.

Category-based behavior differs significantly between the two cities. The top categories (Fig. 6), shown both by total check-ins and unique users,

reveal that rankings not only differ across regions but are also substantially influenced by highly active users. For instance, the 'Office' category ranks high in NYC but is absent from Tokyo's top list. Similarly, bars appear as a top priority for New York users, yet rank at best fifth in Tokyo (after Japanese restaurants). The difference between ranking methods also reveals variation in user activity patterns. Train stations in Tokyo rank first by a large margin (~150K check-ins) when considering total check-ins, but when measured by unique users, they surpass Japanese restaurants by only a few hundred. This suggests that different venue types exhibit distinct user engagement patterns — it is reasonable to assume that a user checking into a train station or subway would do so twice per workday, unlike entertainment venues.

Examining hourly, daily, and monthly activity patterns (Fig. 7), the data shows clear dependence on time of day and month. User activity decreases at night and exhibits pronounced peaks during commuting hours and lunch breaks. The data collection spans from April to February, showing peaks from April to July, followed by a decline in August and September, with no data for October. A second period of increased activity occurs from November to January.

We also examined whether activity patterns vary by category. The row-normalized heatmaps (Fig. 8) show that subway and train stations in both NYC and Tokyo peak in the morning, with NYC users arriving at work shortly after. Bars in New York are popular from after work until dawn, while Tokyo residents frequent ramen shops and Japanese restaurants both for lunch and after-work dinner. Comparing hourly activity between weekdays and weekends (Fig. 13) shows that check-in spikes generally occur on weekdays at similar times in both cities.

Inspecting temporal patterns more closely using a 7-day rolling window (Fig. 12) reveals similar daily check-in patterns for NYC and Tokyo. User activity spikes in the early months of 2012 (April–July), declines from July to October, and then exhibits another spike with a more stable trend line. This may indicate rising app popularity due to advertising campaigns or other external factors; in any case, visible time-based trends are present.

The relationship between categories and the number of unique venues (Fig. 10) appears similar

between NYC and Tokyo, with points concentrated in an almost linear relationship and some anomalous high-outlier points present in both datasets. Comparing category choice diversity among users (Fig. 11), most users fall below 3 bits in both datasets, with medians of 2.9 for NYC and 2.38 for Tokyo.

A minor additional observation: HDBSCAN clustering of POIs (Fig. 9) reveals numerous small clusters, often elongated and likely following street patterns, with both cities exhibiting concentrated urban centers.

Overall, the EDA shows that in both NYC and Tokyo user activity follows power-law distributions. The data exhibits commuting peaks, dramatic weekend shifts and seasonal effects. However between them category preferences diverge sharply, with NYC also winning by the level of user activity and diverse venues and categories. Full code and results can be found in open source [10].

IV. MODELS

This section describes the main model families used in our study. The selected methods represent different levels of complexity in next-POI recommendation: a recurrent sequential model, a graph-enhanced Transformer model, and a global user-graph model. Although the original papers differ in implementation details, they follow a common idea: user mobility is modeled as a sequence of check-ins, while POI, spatial, temporal, and sometimes user-relation information is encoded into dense representations and used to predict the next visited POI.

A. Problem Formulation

Let $\mathcal{U} = \{u_1, u_2, \dots, u_m\}$ be the set of users and let $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ be the set of POIs. Each POI p_i has a unique identifier and geographic coordinates, usually represented by latitude and longitude.

A check-in record is defined as

$$v_i = (u_i, p_i, t_i), \quad (1)$$

where user u_i visits POI p_i at timestamp t_i . For a user u , their mobility history is represented as a time-ordered trajectory

$$T_u = \{v_1, v_2, \dots, v_T\}. \quad (2)$$

The goal of next-POI recommendation is to predict the next POI p_{T+1} given the user's historical trajectory:

$$\hat{p}_{T+1} = \arg \max_{p \in \mathcal{P}} P(p \mid T_u, u). \quad (3)$$

In practice, the model outputs a score for each candidate POI, and the POIs are ranked according to these scores.

B. LSTM-Based Sequential Model

The simplest baseline family is based on recurrent neural networks, especially LSTM models. In next-POI recommendation, the input at each step is usually an embedding of the currently visited POI, optionally combined with temporal and spatial context. The LSTM processes the check-in sequence step by step and produces a hidden state that summarizes the user's recent and historical behavior.

For a check-in at time step t , the input can be written as

$$x_t = e_{p_t} \oplus e_u \oplus e_{\Delta t_t} \oplus e_{\Delta d_t}, \quad (4)$$

where e_{p_t} is the embedding of the visited POI, e_u is the user embedding, Δt_t is the time interval between two consecutive check-ins, Δd_t is the geographic distance between two consecutive POIs, and $\oplus$ denotes concatenation.

The standard LSTM updates its hidden state as

$$h_t, c_t = \text{LSTM}(x_t, h_{t-1}, c_{t-1}), \quad (5)$$

where h_t is the hidden state and c_t is the memory cell. The next POI is predicted from the final hidden state:

$$\hat{y}_{t+1} = \text{softmax}(W_o h_t + b_o). \quad (6)$$

A more advanced version is ST-LSTM [11], which explicitly models the effect of time and distance intervals. The intuition is simple: a visit that happened recently and nearby is usually more important for predicting the next POI than a visit that happened long ago or far away. Therefore, ST-LSTM introduces time-aware and distance-aware gates to control how much information should be kept or forgotten.

Compared with graph-based models, LSTM-based methods mainly model the trajectory as a linear sequence. They are useful as simple and interpretable baselines, but they do not directly capture high-order POI relations, global transition patterns, or collaborative signals from other users.

C. GETNext

GETNext [12] is a graph-enhanced Transformer model for next-POI recommendation. Its main idea is to improve sequential prediction by using a global trajectory flow map. Instead of only considering each user's trajectory independently, GETNext constructs a graph from all user trajectories. In this graph, POIs are nodes, and transitions between consecutive check-ins form directed edges.

Formally, a trajectory transition graph can be written as

$$G_{tr} = (\mathcal{P}, \mathcal{E}_{tr}), \quad (7)$$

where an edge $(p_i, p_j) \in \mathcal{E}_{tr}$ exists if users frequently move from POI p_i to POI p_j . The edge weight may be based on the observed transition frequency:

$$w_{ij} = \frac{\text{count}(p_i \rightarrow p_j)}{\sum_{p_k \in \mathcal{P}} \text{count}(p_i \rightarrow p_k)}. \quad (8)$$

A graph neural network is then used to learn POI embeddings from this global transition structure. A general graph aggregation step can be written as

$$h_i^{(l+1)} = \sigma \left(W^{(l)} h_i^{(l)} + \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W^{(l)} h_j^{(l)} \right), \quad (9)$$

where $h_i^{(l)}$ is the embedding of POI p_i at layer l , $\mathcal{N}(i)$ is the neighborhood of p_i , and α_{ij} controls the importance of neighbor p_j .

After obtaining graph-enhanced POI embeddings, GETNext uses a Transformer to model the user's check-in sequence. The Transformer is better than a plain recurrent model at capturing long-range dependencies because self-attention allows each check-in to directly attend to other check-ins in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V. \quad (10)$$

The main strength of GETNext is that it combines global transition knowledge with sequence modeling. The graph captures common movement patterns shared by many users, while the Transformer captures the order and context of a specific user's trajectory. However, GETNext mainly focuses on POI transition flow and does not explicitly build a global user-user relation graph.

D. GUGEN

GUGEN [13] is a graph-enhanced next-POI recommendation model that combines trajectory modeling with global and user-specific graph structures. The original motivation is that next-POI prediction depends not only on a user’s own recent history, but also on global POI relations, geographic proximity, and recurring POI patterns within a user’s past behavior.

The published implementation consists of five modules:

- **GlobalGraphNet:** a GCN/GAT over a global POI graph encoding POI, category, and discretized GPS features;
- **GlobalDistNet:** a distance graph over POIs capturing geographic relations;
- **UserGraphNet:** a per-user POI graph built from each user’s historical visits;
- **UserHistoryNet:** three GRU branches over POI, category, and temporal features (hour and weekday);
- **TransformerModel:** a 1024-dimensional Transformer that fuses graph signals with trajectory history to predict the next POI.

Formally, the original model combines global graph signals g^{global} , distance graph signals g^{dist} , user graph signals g^{user} , and trajectory history h_t :

$$\hat{y}_{t+1} = \text{Transformer}(h_t, g^{global}, g^{dist}, g^{user}). \quad (11)$$

Training in the original code uses full user trajectories stored as preprocessed pickle files. For a trajectory of length T , the model is trained with cross-entropy loss over *all timesteps*, while evaluation reports $\text{Acc}@k$, $\text{MAP}@k$, and MRR on the last timestep only. On Foursquare-NYC, the original paper reports $\text{Acc}@1 \approx 0.298$, $\text{Acc}@5 \approx 0.593$, and $\text{MRR} \approx 0.383$.

For this project, we do **not** reproduce the full graph-based GUGEN architecture. Instead, we implement a simplified sequential variant, denoted **GuGen-Lite**, designed to run inside a common experimental framework shared with other baselines (e.g., LSTM). The goal is not to match the original paper numbers, but to compare preprocessing strategies and model families under identical data splits, metrics, and training protocols.

Our implementation differs from the original GUGEN in several important ways:

- 1) **Model complexity.** GuGen-Lite removes all graph encoders (global POI graph, distance graph, and per-user graph). It uses POI, user, and category embeddings, a GRU history encoder, a Transformer sequence encoder, and a linear prediction head.
- 2) **Input features.** We use only POI ID, user ID, and category ID. Hour, weekday, and discretized latitude/longitude bins are not included.
- 3) **Sample construction.** Instead of one sample per full trajectory, we build sliding windows of length $L = 20$ with stride 1. Each sample uses 20 consecutive check-ins to predict the next POI.
- 4) **Data split.** We apply a per-user chronological split into train (80%), validation (10%), and test (10%). The original code uses preprocessed train/test pickles without an explicit validation split.
- 5) **Training objective.** We optimize cross-entropy only on the *last position* of each window, while evaluation also uses only the last predicted POI. The original model trains on all timesteps in a trajectory.
- 6) **Preprocessing.** We apply a configurable filter pipeline (minimum visit counts, iterative k -core filtering, optional entropy/dominance filters, and optional geographic bounding boxes for city subsets). The original paper relies on a fixed preprocessing script (`build_trajectory_map.py`) that is not included in our repository.

a) *GuGen-Lite forward pass.*: For a batch of sliding-window samples, let $p_{1:L}$ be the POI sequence, $c_{1:L}$ the category sequence, and u the user ID. We first encode history as

$$h_t = \text{GRU}(e_{p_t} \oplus e_u \oplus e_{c_t}), \quad t = 1, \dots, L, \quad (12)$$

then apply a Transformer encoder and predict the next POI from the final timestep:

$$\hat{p}_{L+1} = \arg \max_{p \in \mathcal{P}} \text{softmax}(W \cdot \text{Transformer}(h_{1:L})_L). \quad (13)$$

Because GuGen-Lite shares the same data loader, evaluation code, checkpointing, and result format as the LSTM baseline, differences in measured performance can be attributed to model capacity and architectural choices rather than preprocessing inconsistencies. Our $\text{Acc}@1$ values (about 0.13–0.22

on Foursquare, depending on filtration) are lower than the original GUGEN result (≈ 0.30 on NYC). This gap is expected: GuGen-Lite omits the graph modules and temporal/geographic features that are central to the published method. Therefore, GuGen-Lite should be interpreted as a *comparative baseline within our framework*, not as a reproduction of the original GUGEN system.

b) GuGen-Graph: simplified graph-enhanced variant.: To incorporate graph neural networks without reproducing the full original preprocessing pipeline, we added another version, a middle-ground model that keeps the unified sliding-window protocol but enriches POI representations with two graphs built from the *training split only*:

- **Transition graph** G_{tr} : directed edges between consecutive POIs observed in training trajectories, weighted by transition frequency;
- **Geo-spatial graph** G_{geo} : undirected edges between POIs whose mean coordinates are within 2 km (Haversine distance), weighted by inverse distance.

Each POI node is initialized from learnable POI/category embeddings plus projected coordinates. Two GCN stacks propagate information on G_{tr} and G_{geo} , and the resulting representations are fused before sequence modeling:

$$e_p = \text{GCN}_{tr}(x_p, G_{tr}) \oplus \text{GCN}_{geo}(x_p, G_{geo}), \quad (14)$$

where x_p is the POI node feature vector. The fused POI embeddings are then fed into the same GRU + Transformer next-POI head as GuGen-Lite.

V. EXPERIMENTS

A. GuGen Experimental Setup

a) Filtration strategies.: We compare five preprocessing settings on the same raw data:

- **no_filter**: minimum 2 visits per user and POI, no k -core;
- **min_10_10**: at least 10 visits per user and POI;
- **min_20_20**: at least 20 visits per user and POI;
- **kcore_10_10**: iterative 10-core filtering on users and POIs;
- **kcore_20_20**: iterative 20-core filtering;

Each experiment is identified by the filter name and stored separately in the results directory.

b) Models: We currently report results for two models trained under the same protocol:

- **GuGen-Lite**: GRU + Transformer, hidden dimension 256, 3 Transformer layers, 8 attention heads;
- **GuGen-Graph**: transition + geo-spatial GCN POI encoder (2 layers each), then GRU + Transformer with the same hidden size and depth as GuGen-Lite;
- **LSTM baseline**: POI/user/category embeddings + 2-layer LSTM, hidden dimension 256, dropout 0.1.

All models are trained with the same general protocol: Adam optimizer, learning rate 5×10^{-4} , weight decay 10^{-5} , batch size 512, sliding-window length 20, and early stopping based on validation Acc@1. The best checkpoint is evaluated on the held-out chronological test split.

We report Acc@1, Acc@5, Acc@10, MAP@5, MAP@10, and MRR. Acc@ k measures whether the correct next POI appears in the top- k recommendations. MAP@ k and MRR additionally account for the rank position of the correct POI.

B. Effect of Filtering

Table VI shows how filtering changes the size and density of each dataset. The effect is especially strong at the POI level. For example, in Foursquare NYC, the `min_20_20` filter removes about 51% of rows but reduces the number of POIs by 94%. Similarly, in Foursquare TKY, the number of POIs decreases from 61,858 in the raw data to 4,025 after `min_20_20`. This confirms that the datasets have a strong long-tail structure: many POIs appear only a small number of times.

The recommendation results in Table III show that stronger filtering generally improves model performance. For GuGen on NYC, Acc@1 increases from 13.38% with no filtering to 18.85% with `min_20_20`. On TKY, GuGen improves from 16.46% with no filtering to 22.45% with `kcore_20_20`. On Gowalla Austin, GuGen improves from 9.11% to 14.27%.

This trend is expected. When rare POIs and extremely sparse interactions are removed, the model sees more repeated behavioral patterns during training. The task becomes less noisy and less affected by one-time visits. However, this also makes the benchmark less realistic, because real-world POI

recommendation systems must often handle rare locations and sparse users. Therefore, filtered results should be interpreted as performance under cleaner and denser conditions, not necessarily as deployment performance on raw mobility data.

C. Comparison Between GuGen and LSTM

The LSTM baseline provides a simpler sequential reference point. It models the user trajectory with recurrent hidden states, but it does not include the Transformer-based sequence encoder used in GuGen.

Across NYC and TKY, GuGen consistently outperforms LSTM under comparable filtering settings. On NYC, the best LSTM result is obtained with `min_20_20`, reaching $\text{Acc}@1 = 15.07\%$, $\text{Acc}@5 = 47.44\%$, $\text{Acc}@10 = 66.65\%$, and $\text{MRR} = 0.3024$. Under the same `min_20_20` filter, GuGen reaches $\text{Acc}@1 = 18.85\%$, $\text{Acc}@5 = 56.34\%$, $\text{Acc}@10 = 73.22\%$, and $\text{MRR} = 0.3548$.

On TKY, the best LSTM result is again obtained with `min_20_20`, reaching $\text{Acc}@1 = 19.73\%$, $\text{Acc}@5 = 53.18\%$, $\text{Acc}@10 = 66.36\%$, and $\text{MRR} = 0.3466$. The best GuGen result on TKY is obtained with `kcore_20_20`, reaching $\text{Acc}@1 = 22.45\%$, $\text{Acc}@5 = 54.96\%$, $\text{Acc}@10 = 67.07\%$, and $\text{MRR} = 0.3694$.

The difference between the two models is moderate but consistent. This suggests that the GRU + Transformer architecture used in GuGen improves ranking quality compared with a plain LSTM, especially on top-1 accuracy and MRR. At the same time, the improvement is not large enough to explain performance alone: preprocessing has a very strong influence for both models.

D. Graph-GuGen Results

Graph-GuGen was introduced to test whether adding explicit POI graph structure improves the simplified GuGen model. Unlike the original GUGEN architecture, our Graph-GuGen uses only two graph views: a transition graph built from consecutive training check-ins and a geo-spatial graph connecting nearby POIs.

The clearest comparison is available for NYC under the `kcore_20_20` filter. As shown in Table V, adding the POI graph improves $\text{Acc}@1$ from 18.30% to 19.10%, $\text{Acc}@5$ from 55.34% to 56.80%, $\text{Acc}@10$ from 73.15% to 74.39%, and MRR from

0.3456 to 0.3551. This is a relatively small but consistent gain across all ranking metrics.

Table IV also shows that Graph-GuGen achieves the best NYC result among all reported models, with $\text{Acc}@1 = 19.10\%$, $\text{Acc}@5 = 56.80\%$, $\text{Acc}@10 = 74.39\%$, and $\text{MRR} = 0.3551$. However, the improvement over GuGen is limited: GuGen with `min_20_20` obtains $\text{Acc}@1 = 18.85\%$ and $\text{MRR} = 0.3548$, which is very close.

For TKY, the available Graph-GuGen runs do not yet cover the strongest `kcore_20_20` configuration. On the reported filters, Graph-GuGen improves over the no-filter GuGen setting, but it does not exceed the best GuGen result on TKY. Therefore, based on the current experiments, the graph module appears useful, but the evidence is still incomplete and should be extended with more runs.

E. Dataset-Level Observations

The results differ substantially across datasets. TKY generally gives the highest $\text{Acc}@1$ values for GuGen, reaching 22.45%. NYC reaches 19.10% with Graph-GuGen and 18.85% with GuGen. Gowalla Austin is harder, with the best reported GuGen $\text{Acc}@1$ equal to 14.27%.

This difference is likely caused by dataset density, city structure, and POI repetition patterns. Gowalla Austin has fewer users in the processed subset and a large number of POIs relative to the number of check-ins, making the candidate space sparse and difficult. In contrast, the filtered Foursquare datasets contain stronger repeated user-POI patterns, especially after applying 20-visit filtering.

The top- k results also show that many errors are near misses rather than completely unrelated predictions. For example, GuGen on NYC with `min_20_20` reaches $\text{Acc}@1 = 18.85\%$, but $\text{Acc}@10 = 73.22\%$. This means that the correct POI often appears among the top recommendations even when it is not ranked first. Similar behavior is observed on TKY, where GuGen with `kcore_20_20` reaches $\text{Acc}@1 = 22.45\%$ and $\text{Acc}@10 = 67.07\%$.

F. Graph-Flashback Experiments

In addition to the LSTM and GuGen-style models, we evaluated a separate family of methods based on Graph-Flashback. These experiments focus on whether graph information can improve next-POI

ranking when it is combined with a Flashback-style sequential model.

Graph-Flashback is different from GuGen in the way graph information is used. Instead of applying a generic POI graph encoder before a Transformer, the Graph-Flashback pipeline constructs a spatio-temporal knowledge graph (STKG), trains TransE embeddings, builds POI transition and user-POI preference graphs, and then combines graph-enhanced representations with a recurrent Flashback predictor. In the enhanced variants, graph and ranking priors are also added directly to the final recommendation scores.

The main evaluated dataset is the Gowalla Austin subset. It contains 195,727 check-ins before downstream filtering, 769 users, 10,077 POIs, optional friendship edges, and approximately 98.4% category coverage. We also ran additional experiments on Foursquare NYC and Foursquare Tokyo under the same min20/20 filtering protocol.

a) Evaluated variants.: We compare the following Graph-Flashback-style methods:

- **Global popularity:** a non-personalized baseline ranking POIs by global frequency;
- **Personal popularity:** a user-specific frequency baseline ranking POIs by the user’s own historical visits;
- **Plain Flashback:** a sequential Flashback baseline without KGE-derived graphs and without additional graph-memory priors;
- **Original-like Graph-Flashback:** a reproduction-oriented version following the paper-style pipeline $STKG \rightarrow TransE \rightarrow transition/user-POI \text{ graphs} \rightarrow RNN + Flashback$;
- **Enhanced Graph-Flashback:** a stronger tuned model using rolling windows, category embeddings, hour-of-week and time-gap embeddings, graph propagation layers, train-only ranking priors, repeat/explore gate, cross-entropy plus BPR loss, and hard negatives;
- **Graph-memory Enhanced Graph-Flashback:** the strongest integrated variant, which adds dynamic graph-memory priors to the enhanced model.

The graph-memory variant is designed to make graph information affect ranking more directly. It uses a dynamic user-history prior based only on locations visited before the target, a transition-graph prior from the last observed POI to graph-

neighbor candidates, learnable prior weights, and a repeat gate. This is useful because graph diagnostics showed that the true next POI is sometimes present in graph neighborhoods, but indirect graph-smoothed embeddings alone may not be strong enough to move it high in the ranking.

G. Graph-Flashback Results

Table VII summarizes the obtained results. On Gowalla Austin, the weakest baseline is global popularity, with $Acc@1 = 0.0136$ and $MRR = 0.0393$. This confirms that a non-personalized ranking is insufficient for next-POI recommendation. Personal popularity is much stronger, reaching $Acc@1 = 0.1021$ and $MRR = 0.1688$. This result is important because it shows that repeat visits are a major part of the Austin mobility pattern.

The original-like Graph-Flashback configuration obtains $Acc@1 = 0.0791$ and $MRR = 0.1232$ on its paper-like/rank protocol. It underperforms the personal popularity baseline on the Austin subset. This does not necessarily contradict the original Graph-Flashback paper, because this configuration is reproduction-oriented and uses a different subset and protocol. It also uses smaller and less tuned settings. In our experiments, simply reproducing the paper-style graph pipeline was not enough to outperform a strong repeat-visit baseline on the Austin subset.

The enhanced version performs better. On the common Austin split, Enhanced Graph-Flashback reaches $Acc@1 = 0.1149$ and $MRR = 0.1852$, improving over personal popularity. On the stricter min20/20 protocol, it further improves to $Acc@1 = 0.1329$ and $MRR = 0.2117$. This again confirms the strong influence of filtering: removing rare POIs and sparse interactions makes the task more predictable and improves ranking quality.

The best Gowalla Austin result is achieved by Graph-memory Enhanced Graph-Flashback on the min20/20 protocol. It reaches $Acc@1 = 0.1422$, $Acc@5 = 0.2932$, $Acc@10 = 0.3740$, $MAP@5 = 0.1974$, $MAP@10 = 0.2081$, and $MRR = 0.2201$. Compared with Enhanced Graph-Flashback on the same min20/20 split, graph memory improves $Acc@1$ by 0.0093 and MRR by 0.0084. The improvement is not extremely large, but it is consistent across all metrics. This suggests that direct graph-memory priors help the model rank plausible next POIs higher.

The Foursquare NYC results are substantially higher than the Austin results. Plain Flashback already reaches $\text{Acc}@1 = 0.2973$ and $\text{MRR} = 0.4415$. Enhanced Graph-Flashback improves this to $\text{Acc}@1 = 0.3143$ and $\text{MRR} = 0.4547$. The graph-memory version gives the best NYC performance, with $\text{Acc}@1 = 0.3206$, $\text{Acc}@5 = 0.6492$, $\text{Acc}@10 = 0.7378$, and $\text{MRR} = 0.4642$. This shows that graph and prior-based enhancements are beneficial even when the sequential baseline is already strong.

On Foursquare Tokyo, the pattern is slightly different. Enhanced Graph-Flashback achieves the highest $\text{Acc}@1$, 0.2586, while Graph-memory Enhanced Graph-Flashback achieves the best $\text{Acc}@5$, $\text{Acc}@10$, $\text{MAP}@5$, $\text{MAP}@10$, and MRR . Its MRR is 0.3860 compared with 0.3831 for Enhanced Graph-Flashback and 0.3729 for Plain Flashback. This means that graph memory improves the overall ranking quality, although it does not always place the correct POI at rank 1.

The paper-reference rows are included only as external context. They are not directly comparable with our Austin subset experiments because the original paper uses the full processed Gowalla dataset and a different evaluation protocol. Still, they are useful as a rough indication of expected performance ranges for Flashback and Graph-Flashback-style models.

VI. CONCLUSION

This project studied next-POI recommendation under unified preprocessing and evaluation conditions, with special attention to graph-based modeling, spatial information, sparsity, and temporal behavior. We evaluated several model families: a simple LSTM baseline, a simplified GuGen-style Transformer model, a Graph-GuGen variant with POI graphs, and several Graph-Flashback-style models.

The first important finding is that preprocessing has a major influence on performance. Across GuGen, LSTM, and Graph-Flashback experiments, stronger filtering generally improves $\text{Acc}@k$, $\text{MAP}@k$, and MRR . For example, GuGen on Gowalla Austin improves from $\text{Acc}@1 = 9.11\%$ with weak filtering to 14.27% with min20/20 or kcore20/20 filtering. Similarly, Enhanced Graph-Flashback improves from $\text{Acc}@1 = 0.1149$ on the common Austin split to 0.1329 on the min20/20 split. This confirms that raw LBSN data are highly

sparse and long-tailed, and that reported benchmark performance depends strongly on filtering choices.

The second finding is that stronger sequence modeling improves over a plain recurrent baseline. In the GuGen-style experiments, the Transformer-based GuGen model consistently outperforms the LSTM baseline on Foursquare NYC and Tokyo. On NYC, GuGen reaches $\text{Acc}@1 = 18.85\%$ and $\text{MRR} = 0.3548$, while LSTM reaches $\text{Acc}@1 = 15.07\%$ and $\text{MRR} = 0.3024$ under the same min20/20 filtering. This shows that attention-based sequence modeling can capture useful longer-range dependencies beyond a standard LSTM.

The third finding is that graph information is useful, but its effect depends on how it is connected to prediction. In the GuGen-style experiments, Graph-GuGen improves over GuGen on NYC under the same kcore20/20 setting, increasing $\text{Acc}@1$ from 18.30% to 19.10% and MRR from 0.3456 to 0.3551. This confirms that transition and geo-spatial POI graphs provide additional signal. However, the gain is moderate, suggesting that graph-smoothed embeddings alone may not be sufficient.

The Graph-Flashback experiments support this interpretation. The strongest model is Graph-memory Enhanced Graph-Flashback, which makes graph information affect the final ranking directly through dynamic user-history and transition-graph priors. On Gowalla Austin min20/20, it achieves $\text{Acc}@1 = 0.1422$, $\text{Acc}@10 = 0.3740$, and $\text{MRR} = 0.2201$. On Foursquare NYC min20/20, it obtains the best overall result among our experiments, with $\text{Acc}@1 = 0.3206$, $\text{Acc}@10 = 0.7378$, and $\text{MRR} = 0.4642$. On Foursquare Tokyo min20/20, it gives the best MRR , $\text{Acc}@5$, $\text{Acc}@10$, $\text{MAP}@5$, and $\text{MAP}@10$, although Enhanced Graph-Flashback has slightly higher $\text{Acc}@1$.

A notable observation is that personal popularity is already a strong baseline on Gowalla Austin, reaching $\text{Acc}@1 = 0.1021$ and $\text{MRR} = 0.1688$. This means that repeat visits are central in POI recommendation, and graph models must outperform not only simple global popularity but also strong personalized frequency baselines. The original-like Graph-Flashback reproduction did not outperform personal popularity on Austin, while the enhanced and graph-memory variants did. This shows the importance of tuning, ranking losses, and direct priors in addition to graph construction.

Overall, the experiments suggest that modern POI

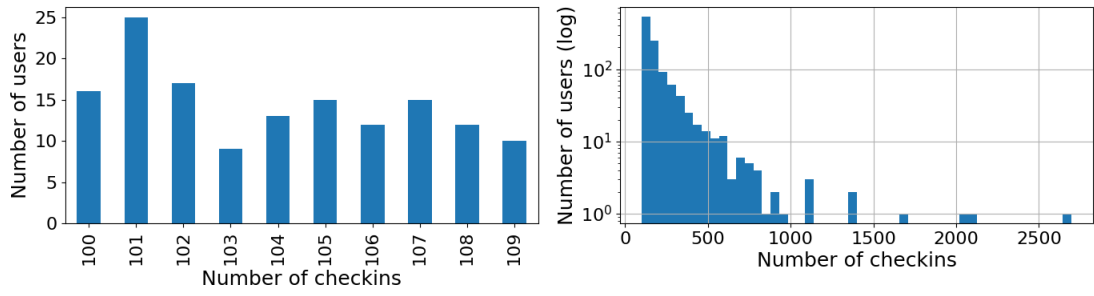
recommendation performance is driven by three factors: dataset filtering, personalized repeat behavior, and the way graph information is used in ranking. Graph neural networks and graph-derived relations are helpful, but the strongest gains appear when graph signals are combined with temporal sequence modeling and direct ranking priors.

Future work should extend the robustness part of the project more explicitly. In particular, the next step is to apply controlled spatial perturbations to coordinates and graph edges, evaluate temporal drift using stricter chronological splits, and compare how LSTM, GuGen-style models, and Graph-Flashback models degrade under noise. It would also be useful to test whether graph-memory priors remain stable when GPS coordinates are perturbed or when venue popularity changes over time.

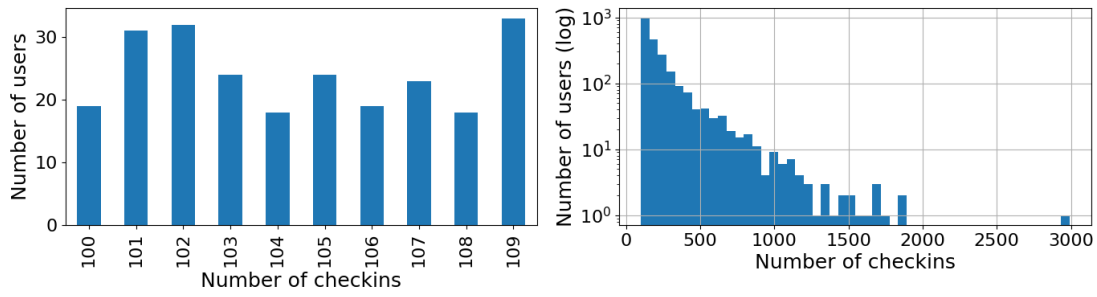
REFERENCES

- [1] J. Wu, X. Wang, F. Feng, X. He, L. Chen, J. Lian, and X. Xie, "Self-supervised graph learning for recommendation," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 726–735.
- [2] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and Q. V. H. Nguyen, "Are graph augmentations necessary? simple graph contrastive learning for recommendation," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022, pp. 1294–1303.
- [3] Z. Lin, C. Tian, Y. Hou, and W. X. Zhao, "Improving graph collaborative filtering with neighborhood-enriched contrastive learning," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 2320–2329.
- [4] H. Zhou, Z. Jia, H. Zhu, and Z. Zhang, "CLLP: Contrastive learning framework based on latent preferences for next POI recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 1473–1482.
- [5] Y. Wang and Q. Jiang, "Enhancing next POI recommendation via multi-graph modeling and multi-granularity contrastive learning," *Journal of King Saud University - Computer and Information Sciences*, vol. 37, p. 343, 2025.
- [6] D. Yang, D. Zhang, V. W. Zheng, and Z. Yu, "Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 45, no. 1, pp. 129–142, 2015, <https://sites.google.com/site/yangdingqi/home/foursquare-dataset>.
- [7] E. Cho, S. A. Myers, and J. Leskovec, "Friendship and mobility: User movement in location-based social networks," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2011, pp. 1082–1090.
- [8] "gowalla_data," https://figshare.com/articles/dataset/gowalla_data/22126586/1, 2023, figshare dataset, accessed May 22, 2026.
- [9] M. Urazmetova, "Full gowalla exploratory data analysis with figshare POI metadata," https://colab.research.google.com/drive/1vbVnU8Fj_wbxYSYV9KXu_IGtEu3dQtBD?usp=sharing, 2026, supplementary notebook.
- [10] U. Klyuchnikova, "Full foursquare tkynyc exploratory data analysis," <https://www.kaggle.com/code/alice6fox/foursquaregraphs-eda>, 2026, supplementary notebook.
- [11] P. Zhao, H. Zhu, Y. Liu, Z. Li, J. Xu, and V. S. Sheng, "Where to go next: A spatio-temporal lstm model for next poi recommendation," 2018. [Online]. Available: <https://arxiv.org/abs/1806.06671>
- [12] S. Yang, J. Liu, and K. Zhao, "GETNext: Trajectory flow map enhanced transformer for next POI recommendation," *arXiv preprint arXiv:2303.04741*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.04741>
- [13] C. Zuo, X. Zhang, L. Yan, and Z. Zhang, "GUGEN: Global user graph enhanced network for next POI recommendation," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 14 975–14 986, 2024.

APPENDIX

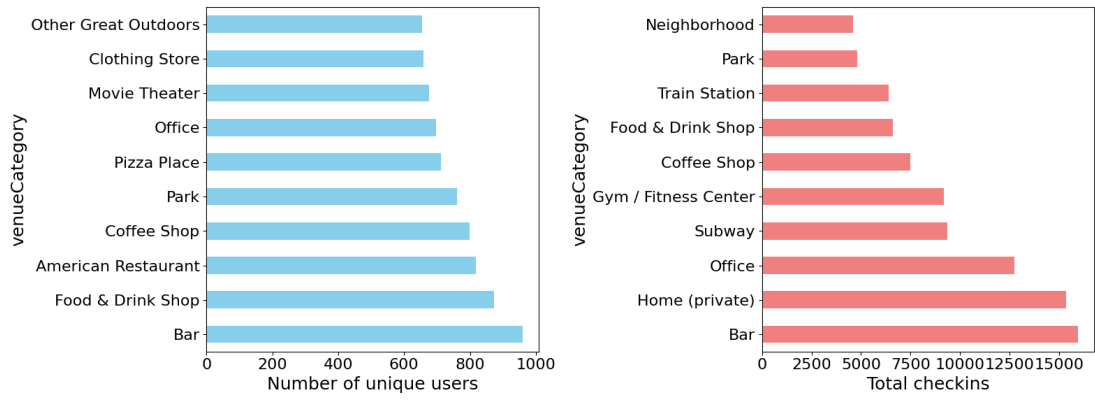


(a) NYC: Check-ins per user

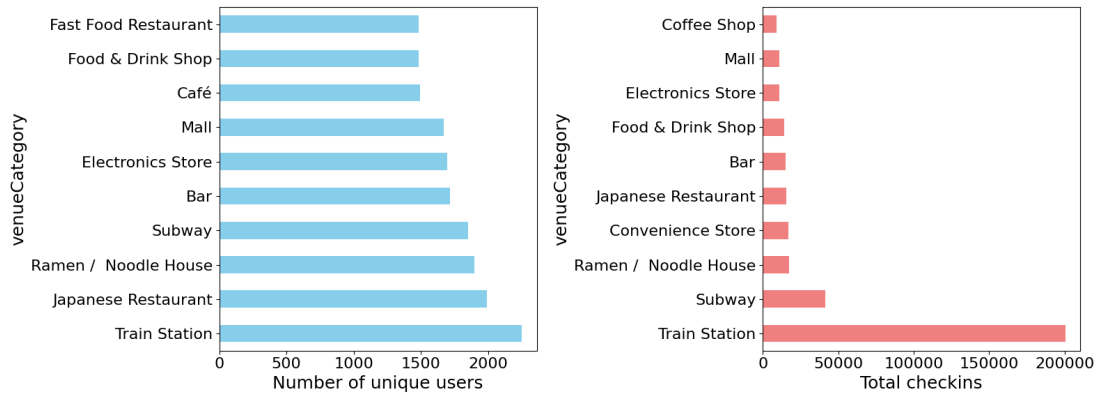


(b) Tokyo: Check-ins per user

Fig. 5. Distribution of user activity levels. Left: Top-10 users by check-in count. Right: Log-scale histogram showing power-law behavior.

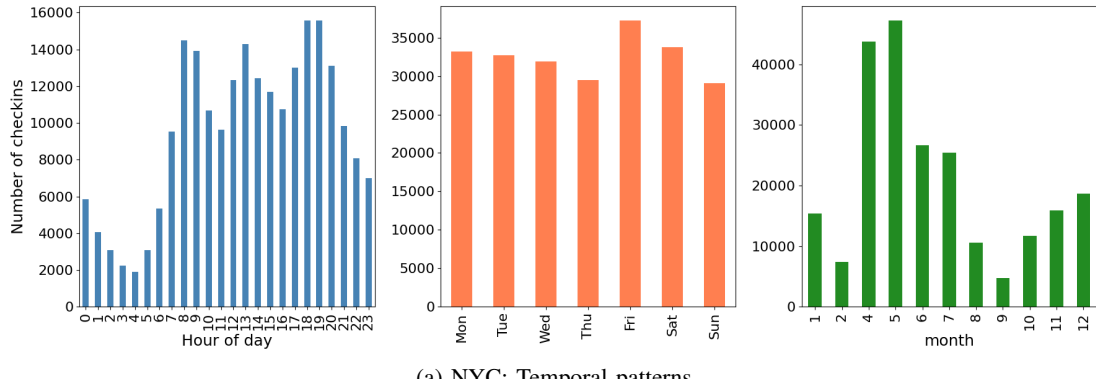


(a) NYC: Top categories

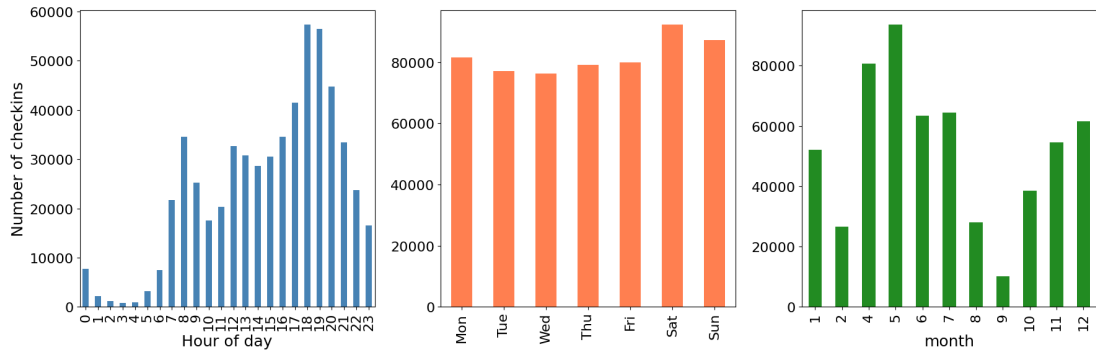


(b) Tokyo: Top categories

Fig. 6. Top-10 venue categories. Left: By unique users (reach). Right: By total check-ins (frequency).

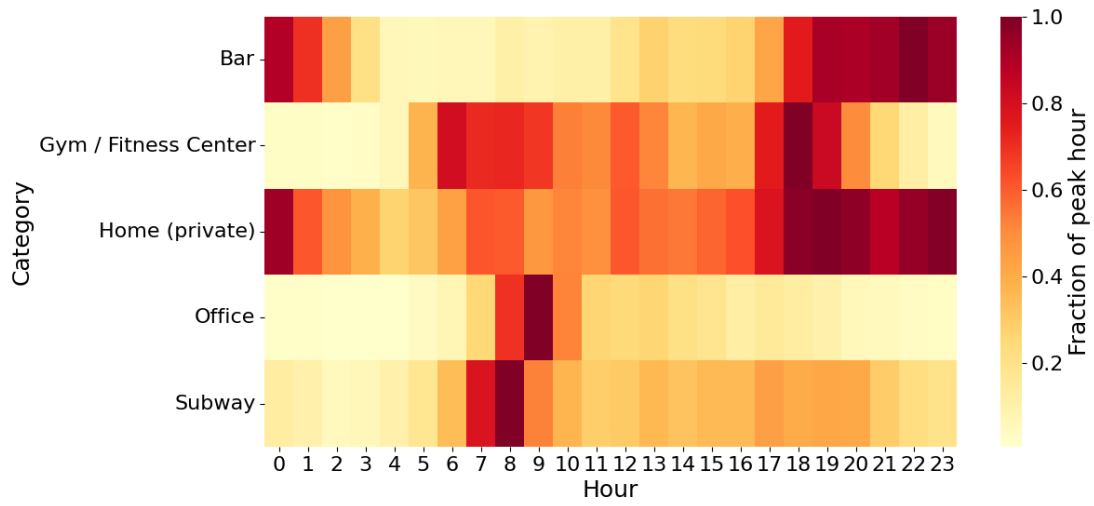


(a) NYC: Temporal patterns

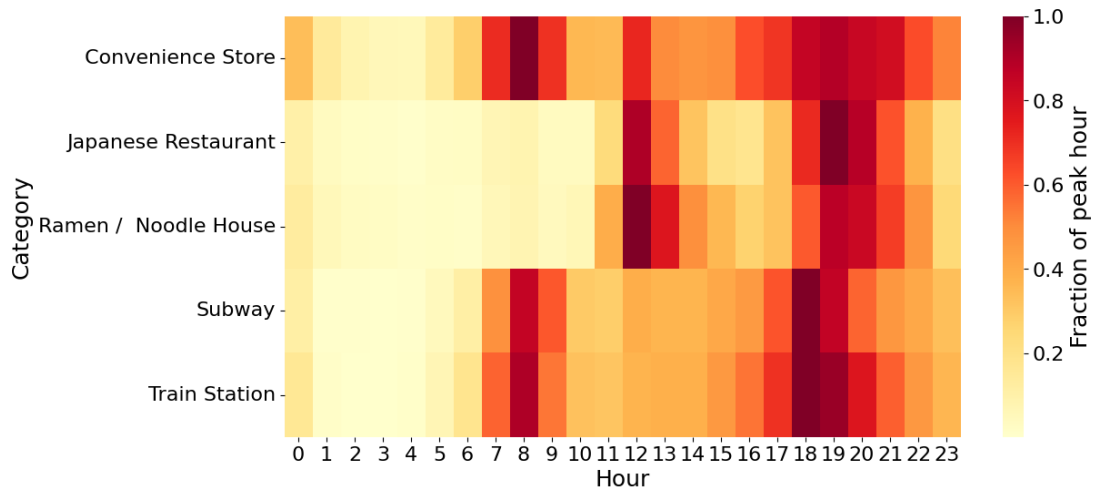


(b) Tokyo: Temporal patterns

Fig. 7. Temporal patterns: Hourly, daily, and monthly distributions.



(a) NYC: Category-hour heatmap



(b) Tokyo: Category-hour heatmap

Fig. 8. Category activity by hour (row-normalized). Darker color = peak hour for that category.

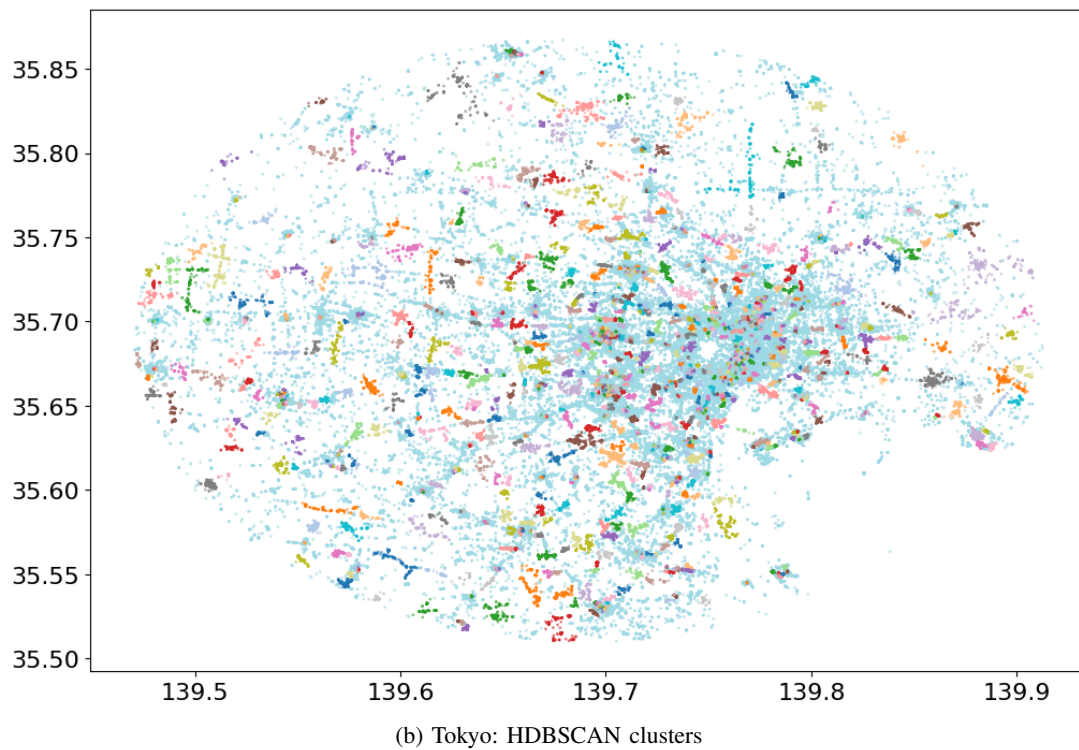
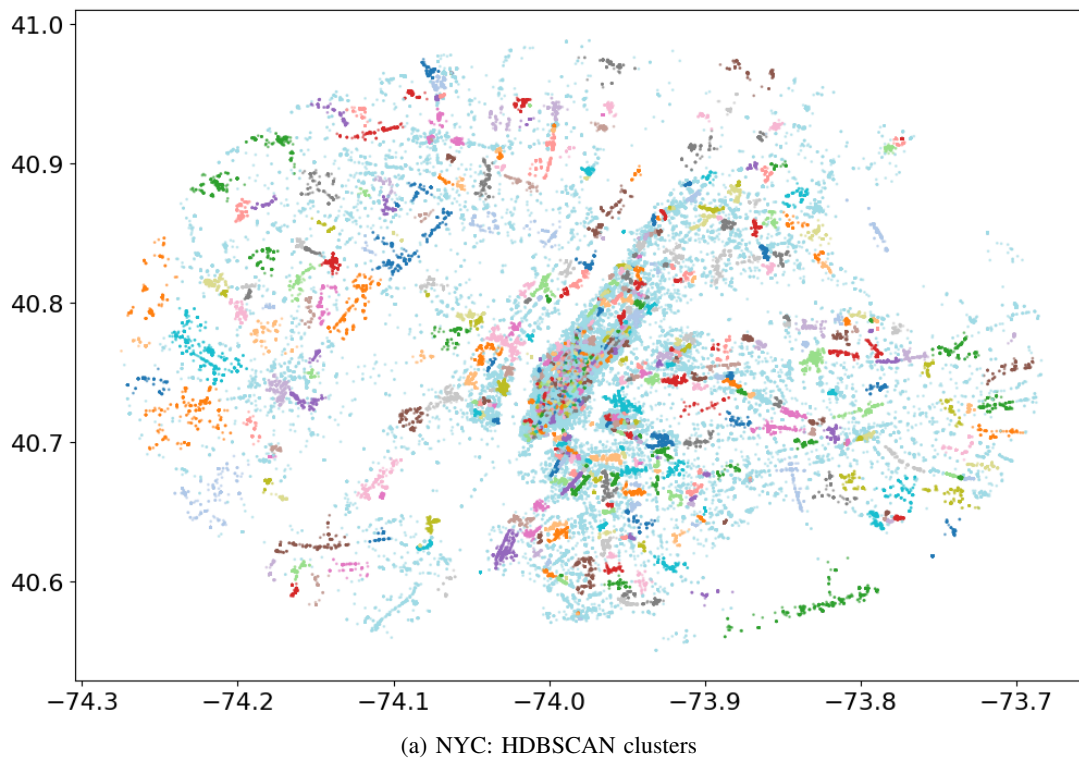
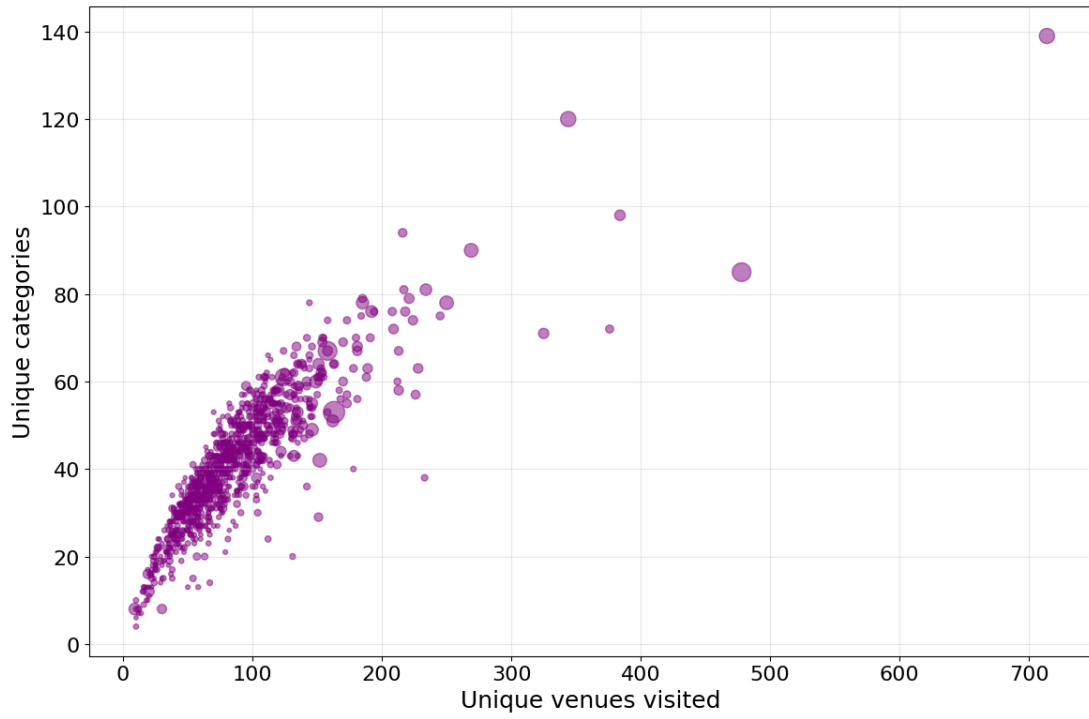
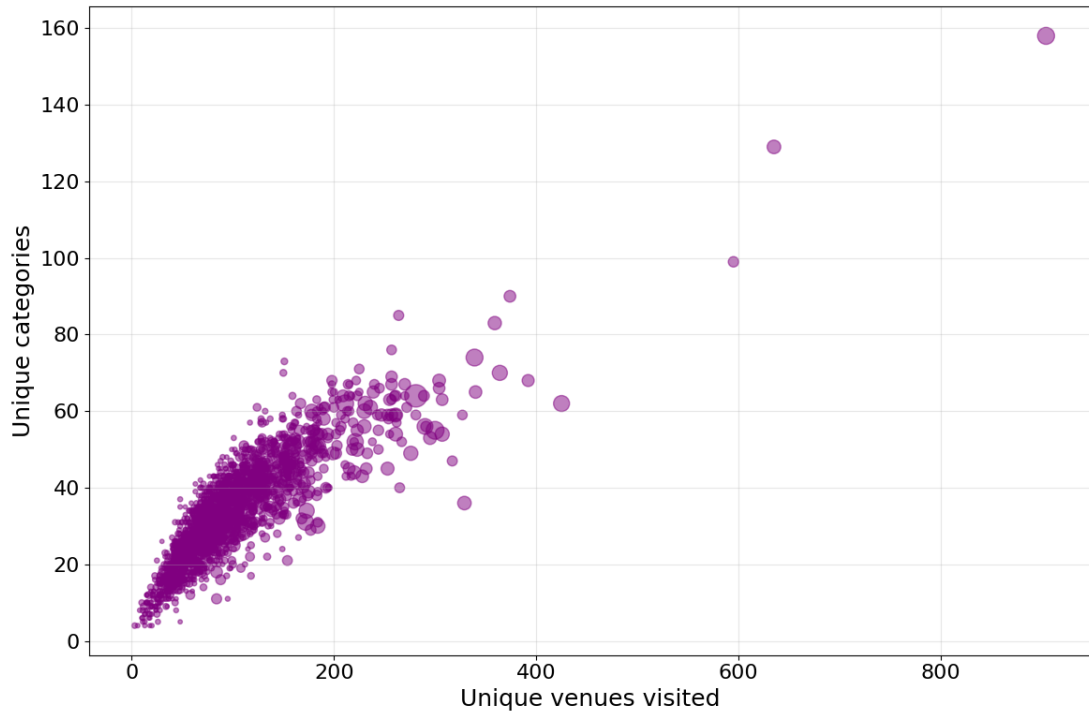


Fig. 9. Geographic clustering of check-ins. Each color represents a distinct cluster; gray points are noise.

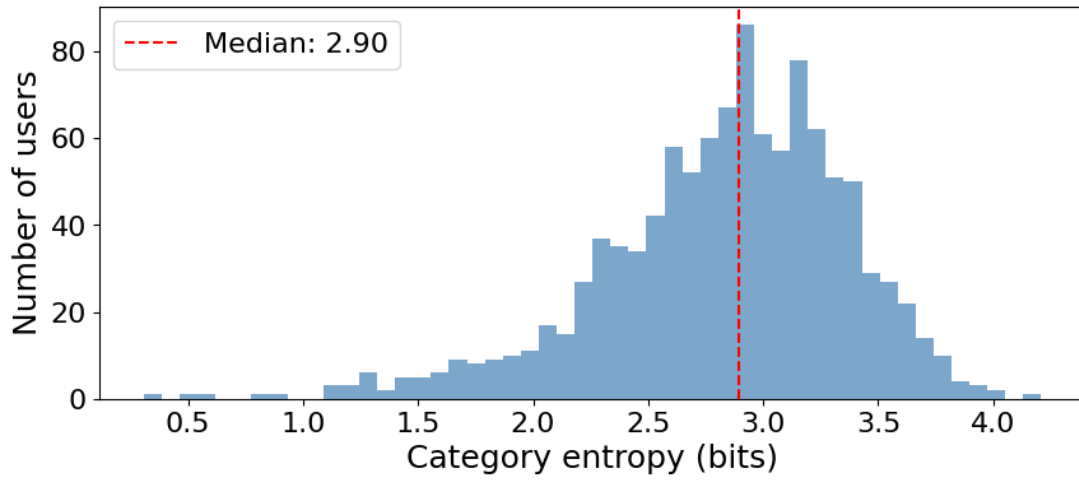


(a) NYC: Venues vs Categories

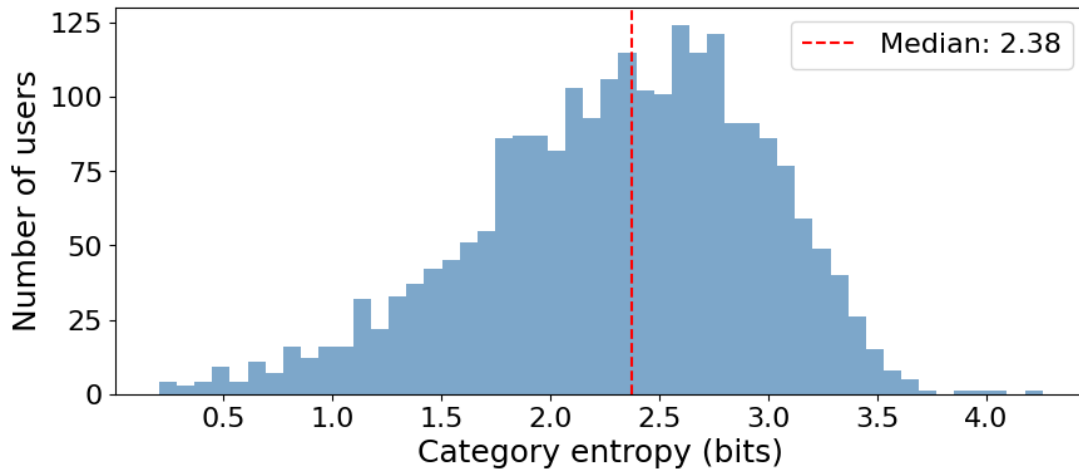


(b) Tokyo: Venues vs Categories

Fig. 10. User diversity: Unique venues visited vs unique categories explored. Point size = total check-ins.

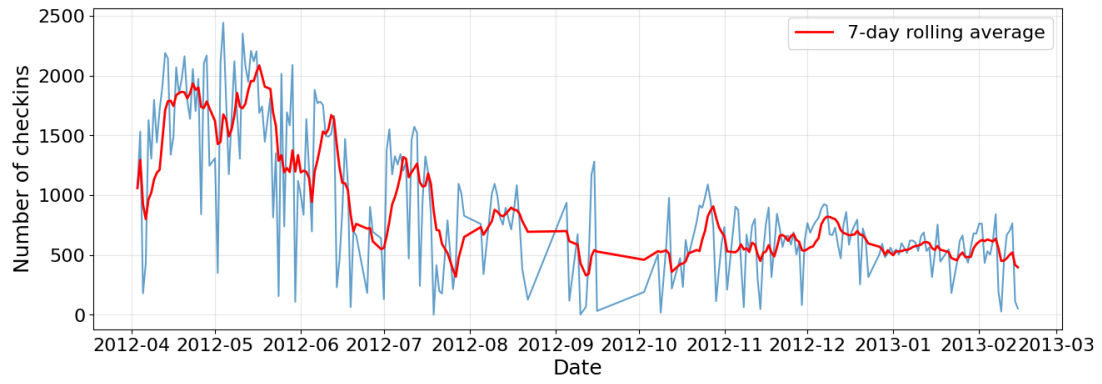


(a) NYC: Category entropy

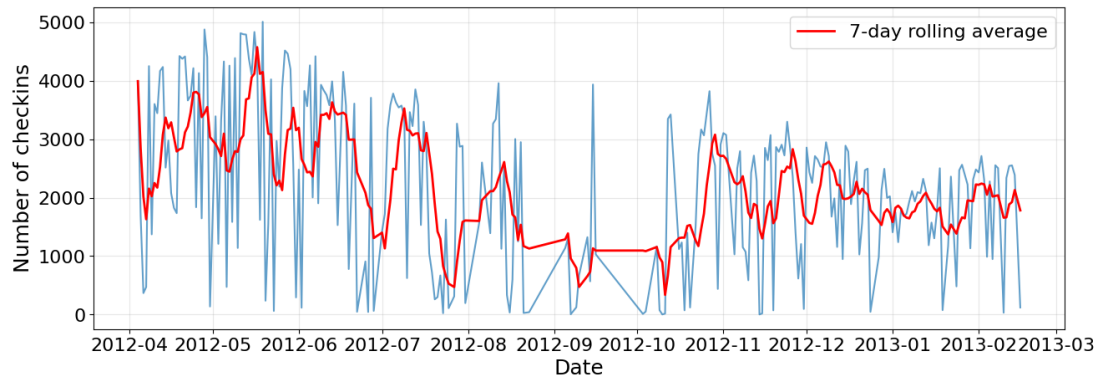


(b) Tokyo: Category entropy

Fig. 11. User category diversity measured by Shannon entropy (bits). Higher entropy = more diverse exploration.

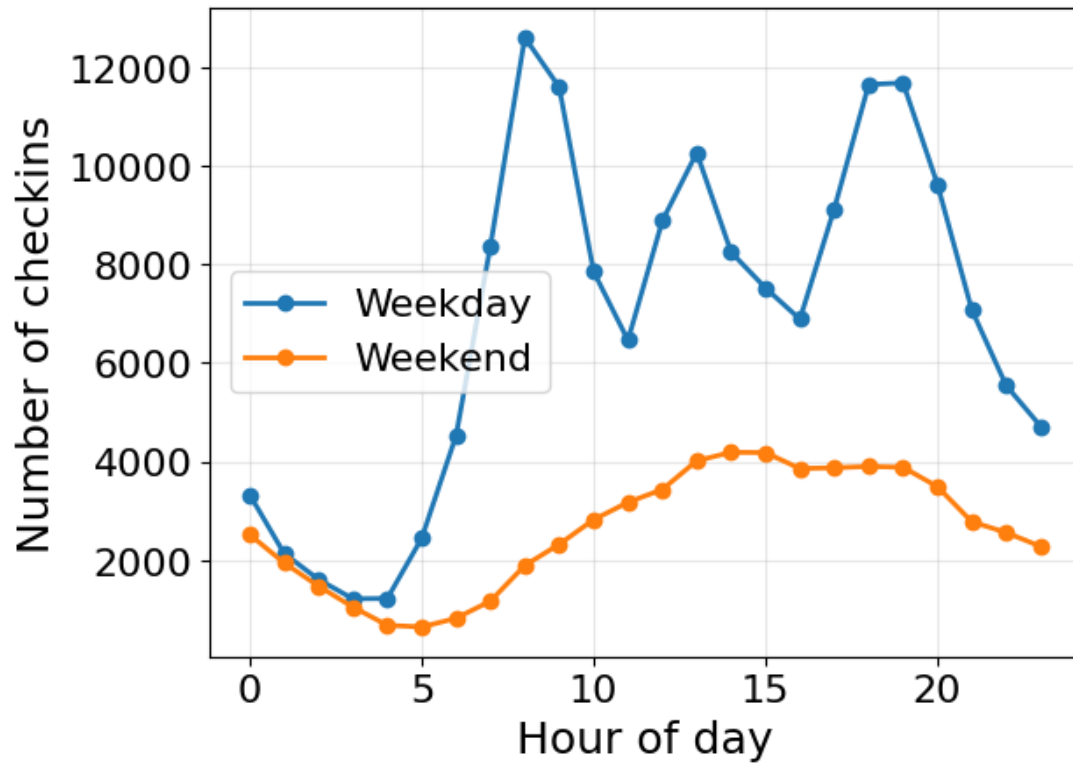


(a) NYC: Daily check-ins

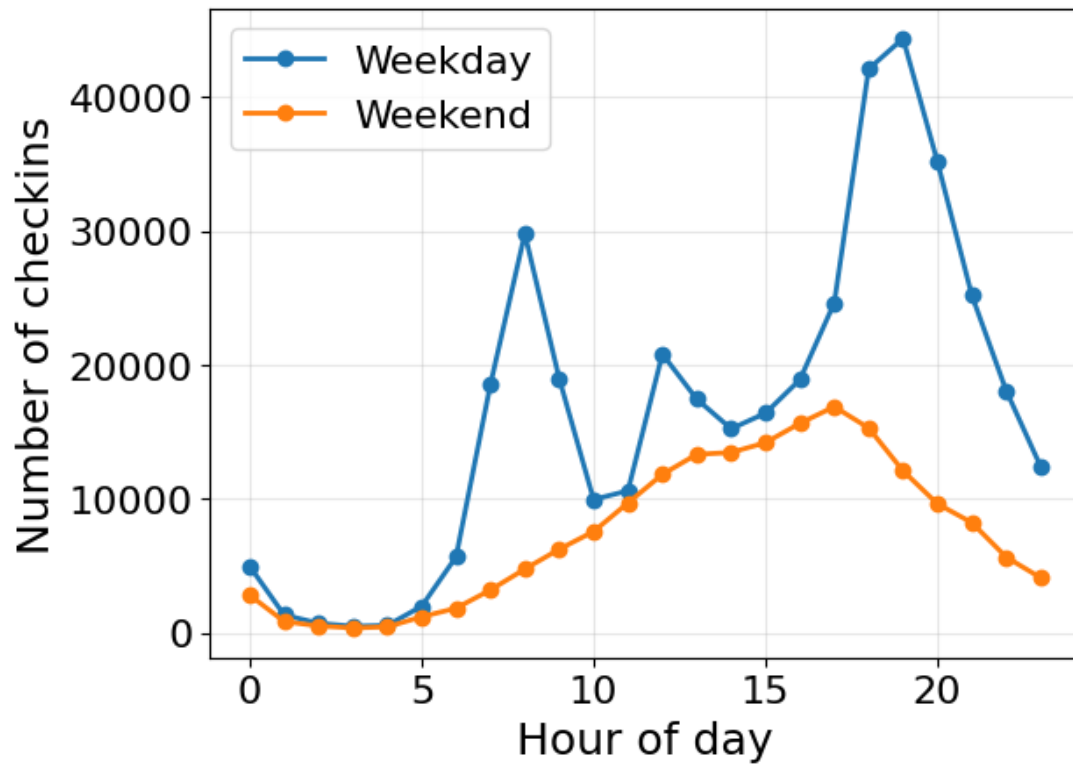


(b) Tokyo: Daily check-ins

Fig. 12. Daily check-ins with 7-day rolling average (red).



(a) NYC: Weekday vs Weekend



(b) Tokyo: Weekday vs Weekend

Fig. 13. Weekday vs weekend hourly patterns.

TABLE III
POI RECOMMENDATION RESULTS: GUGEN VS LSTM WITH DIFFERENT FILTRATION STRATEGIES

Model	Dataset	Filter	ACC@1	ACC@5	ACC@10	MAP@5	MAP@10	MRR
GuGen (Transformer-based)								
GuGen	NYC	min,20,20	18.85%	56.34%	73.22%	0.3212	0.3441	0.3548
		k,20,20	18.30%	55.34%	73.15%	0.3113	0.3353	0.3456
		min,10,10	14.81%	43.70%	62.04%	0.2510	0.2756	0.2885
		k,10,10	16.80%	48.86%	65.47%	0.2796	0.3018	0.3137
		none	13.38%	36.73%	48.11%	0.2157	0.2312	0.2424
GuGen	TKY	k,20,20	22.45%	54.96%	67.07%	0.3430	0.3596	0.3694
		min,20,20	21.37%	54.56%	67.38%	0.3355	0.3529	0.3626
		min,10,10	19.49%	48.49%	60.27%	0.2999	0.3159	0.3266
		k,10,10	19.49%	48.49%	60.27%	0.2999	0.3159	0.3266
		none	16.46%	41.36%	51.42%	0.2543	0.2679	0.2783
GuGen	Gowalla	k,20,20	14.27%	30.26%	38.26%	0.2003	0.2110	0.2227
		min,20,20	14.27%	30.26%	38.26%	0.2003	0.2110	0.2227
		min,10,10	12.72%	25.75%	33.09%	0.1743	0.1840	0.1950
		none	9.11%	19.99%	25.83%	0.1305	0.1384	0.1476
Graph-GuGen (with POI Graph)								
	NYC	k,20,20	19.10%	56.80%	74.39%	0.3208	0.3448	0.3551
Graph-GuGen		min,10,10	20.65%	50.50%	62.48%	0.3154	0.3316	0.3425
Graph-GuGen	TKY	none	17.13%	41.83%	51.99%	0.2613	0.2750	0.2856
LSTM (RNN-based)								
LSTM	NYC	min,20,20	15.07%	47.44%	66.65%	0.2627	0.2884	0.3024
		k,20,20	14.44%	48.21%	66.37%	0.2618	0.2857	0.2987
		min,10,10	13.56%	41.20%	57.84%	0.2324	0.2547	0.2695
		k,10,10	13.56%	41.20%	57.84%	0.2324	0.2547	0.2695
		none	11.61%	32.51%	45.84%	0.1910	0.2090	0.2219
LSTM	TKY	min,20,20	19.73%	53.18%	66.36%	0.3191	0.3370	0.3466
		k,20,20	19.65%	52.88%	66.09%	0.3168	0.3348	0.3447
		min,10,10	17.87%	47.89%	60.83%	0.2878	0.3052	0.3159
		none	15.75%	40.81%	51.29%	0.2483	0.2625	0.2727

TABLE IV
BEST RESULTS: GUGEN VS GRAPH-GUGEN VS LSTM

Model	Dataset	Best Filter	ACC@1	ACC@5	ACC@10	MAP@5	MAP@10	MRR
Graph-GuGen	NYC	k,20,20	19.10%	56.80%	74.39%	0.3208	0.3448	0.3551
GuGen	NYC	min,20,20	18.85%	56.34%	73.22%	0.3212	0.3441	0.3548
GuGen	TKY	k,20,20	22.45%	54.96%	67.07%	0.3430	0.3596	0.3694
LSTM	TKY	min,20,20	19.73%	53.18%	66.36%	0.3191	0.3370	0.3466
GuGen	Gowalla	k,20,20	14.27%	30.26%	38.26%	0.2003	0.2110	0.2227

TABLE V
IMPACT OF POI GRAPH ON GU_{GEN} PERFORMANCE

Dataset	Filter	Model	ACC@1	ACC@5	ACC@10	MAP@5	MAP@10	MRR	Improvement
NYC	k,20,20	GuGen	18.30%	55.34%	73.15%	0.3113	0.3353	0.3456	—
		Graph-GuGen	19.10%	56.80%	74.39%	0.3208	0.3448	0.3551	+0.80%
TKY	min,10,10	GuGen	19.49%	48.49%	60.27%	0.2999	0.3159	0.3266	—
		Graph-GuGen	20.65%	50.50%	62.48%	0.3154	0.3316	0.3425	+1.16%
TKY	none	GuGen	16.46%	41.36%	51.42%	0.2543	0.2679	0.2783	—
		Graph-GuGen	17.13%	41.83%	51.99%	0.2613	0.2750	0.2856	+0.67%

TABLE VI
DATASET STATISTICS AFTER DIFFERENT FILTRATION STRATEGIES

Dataset	Filter	Rows	Users	POIs	Categories	Train Seq
Foursquare NYC (Raw: 227,428 rows, 1,083 users, 38,333 POIs, 400 categories)						
NYC	raw	227,428	1,083	38,333	400	159,832
	no_filter	210,845 (-7%)	1,083	21,750 (-43%)	388	146,583
	min_10_10	147,938 (-35%)	1,083	5,135 (-87%)	321	96,284
	min_20_20	111,457 (-51%)	1,083	2,413 (-94%)	253	67,737
	kcore_20_20	110,394 (-51%)	1,030 (-5%)	2,391	253	67,438
Foursquare TKY (Raw: 574,000 rows, 2,293 users, 61,858 POIs)						
TKY	no_filter	546,622 (-5%)	2,293	34,777 (-44%)	—	390,510
	min_10_10	447,570 (-22%)	2,293	7,873 (-87%)	—	311,298
	min_20_20	395,564 (-31%)	2,293	4,025 (-93%)	—	269,988
	kcore_20_20	394,886 (-31%)	2,264 (-1%)	4,011	—	269,780
Gowalla Austin (Raw: 196,000 rows, 769 users, 10,077 POIs)						
Gowalla	no_filter	195,727 (0%)	769	10,077 (0%)	—	140,901
	min_10_10	165,305 (-16%)	769	3,909 (-61%)	—	116,558
	min_20_20	141,332 (-28%)	769	2,127 (-79%)	—	97,384

TABLE VII
GRAPH-FLASHBACK RESULTS ON GOWALLA AUSTIN AND FOURSQUARE NYC/TOKYO

Model	Dataset / Protocol	Acc@1	Acc@5	Acc@10	MAP@5	MAP@10	MRR
Global popularity	Gowalla Austin common split	0.0136	0.0519	0.0846	0.0288	0.0333	0.0393
Personal popularity	Gowalla Austin common split	0.1021	0.2381	0.3044	0.1513	0.1600	0.1688
Original-like Graph-Flashback	Gowalla Austin paper-like/rank	0.0791	0.1641	0.2032	0.1105	0.1158	0.1232
Enhanced Graph-Flashback	Gowalla Austin common split	0.1149	0.2507	0.3214	0.1643	0.1737	0.1852
Enhanced Graph-Flashback	Gowalla Austin min20/20	0.1329	0.2882	0.3631	0.1898	0.1998	0.2117
Graph-memory Enhanced Graph-Flashback	Gowalla Austin min20/20	0.1422	0.2932	0.3740	0.1974	0.2081	0.2201
Plain Flashback	Foursquare NYC min20/20	0.2973	0.6252	0.7100	0.4238	0.4353	0.4415
Enhanced Graph-Flashback	Foursquare NYC min20/20	0.3143	0.6384	0.7253	0.4366	0.4485	0.4547
Graph-memory Enhanced Graph-Flashback	Foursquare NYC min20/20	0.3206	0.6492	0.7378	0.4460	0.4580	0.4642
Plain Flashback	Foursquare Tokyo min20/20	0.2444	0.5298	0.6300	0.3505	0.3640	0.3729
Enhanced Graph-Flashback	Foursquare Tokyo min20/20	0.2586	0.5342	0.6347	0.3605	0.3741	0.3831
Graph-memory Enhanced Graph-Flashback	Foursquare Tokyo min20/20	0.2568	0.5427	0.6457	0.3628	0.3768	0.3860
Flashback, paper reference	Full Gowalla	0.1158	0.2754	0.3479	—	—	0.1925
Graph-Flashback, paper reference	Full Gowalla	0.1512	0.3425	0.4256	—	—	0.2422